

Taste Engine

Neighborhood-aware culinary intelligence. Turning local taste signals into operational menu strategy for fast-casual brands.

[Live dashboard](#) · [Demo video](#) · [Source on GitHub](#)

What it is

Taste Engine is a **quarterly intelligence brief for brand strategy teams**. It reads the local food culture of a single neighborhood — restaurant menus, review mentions, indie cuisine mix — surfaces flavors where national trend strength and local review signal agree, flags maturity transitions (when flavors cross from chef-driven to mainstream), and outputs menu recommendations grounded in each brand's actual pantry (active ingredient inventory), operational lift capacity, and limited-time offer (LTO) history. Every visit shows what changed since last quarter and a directional forecast for the next.

Demo coverage: **Chipotle, CAVA, Sweetgreen** across **West Hollywood, Williamsburg (Brooklyn), Mission District (San Francisco)**.

The output is two recommendation types per brand per neighborhood: - **Ship Now** — dishes the brand can execute immediately from current SKUs (stock units — any single inventory item a brand stocks) - **Gap Fill** — high-signal flavors that require specific SKU additions, with named operational lift

Every recommendation passes a deterministic deliverability check before being labeled novel. Every recommendation carries a composite confidence score (0–100), a trend maturity stage, an operational lift tier, a rollout portability tag, and — where applicable — a "PROVEN" badge linking to newsroom verification that the brand has shipped the flavor before.

Why now

Fast-casual menu innovation cycles are compressing. Chipotle accelerated from two limited-time proteins per year to three or four. CAVA does four full seasonal menu changes annually. Sweetgreen rebuilt its dinner platform in one cycle. National brands feel local irrelevance pressure as TikTok and social food media accelerate regional flavor diffusion — by the time a chain's R&D pipeline ships a trend, the originating neighborhood has moved on.

The bottleneck is not creativity. It is *intelligence*. Brands need to know which flavors are mainstream in a specific neighborhood (safe shipping window) versus still chef-driven (watch market, premature ship), and to know it from real local data — not training-data stereotypes about "what hipster Brooklyn likes."

Taste Engine builds that intelligence layer.

The quarterly cadence

Brands run quarterly menu planning. Taste Engine fits that cadence: every visit opens with a brief showing **what changed since last quarter** (top movers up, top movers down, flavors that crossed maturity thresholds) and a directional **next-quarter forecast** for the top flavors. Examples from the current Mission District brief:

- **Movers up:** mole +7, al pastor +6, pesto +5 mentions vs prior quarter
- **Maturity transitions:** al pastor crossed Weak → Established between Q4 2025 and Q1 2026 (right before Chipotle's Feb 2026 relaunch announcement)
- **Forecast:** birria projected to rise to ~27 mentions in Q2 (± 2), continuing through Q3

This is the difference between a one-time analysis and a subscription. Operators come back to Taste Engine the way they come back to a sales dashboard — for the deltas.

What rolls out as the platform accrues longitudinal data

Two capabilities are honest "v2 features" we are surfacing intentionally as roadmap:

- **Multi-snapshot Google Trends history.** Today the trend signal uses a 12-month rolling average per pull. Switching to an append-only `trends_history` table with weekly cron snapshots produces real velocity series — distinguishing flavors that are rising fast from flavors that have been steady for years.
- **Cross-city flow predictions.** When pesto becomes Established in WeHo, Rising in Mission, and Steady in Williamsburg, we have a cultural diffusion signal. With ~12 quarters of multi-city data we can fit lag patterns ("pesto moves east, ~6 quarters behind LA") and forecast which neighborhood adopts a flavor next. Currently we have 8 quarters; another year of monthly snapshots makes this defensible.

Both are architecturally trivial — the temporal tables already exist; we just have to accumulate the snapshots. Neither blocks the demo today.

Three pillars

Pillar 1 — Signal Fusion

National Google Trends at the metro market level (LA Metro 803, SF-Oakland 807, NYC 501) cross-referenced with local indie restaurant reviews, weighted honestly. The system surfaces flavors where both signals agree, and explicitly flags the cases where they disagree (local-led demand without national buzz, or national hype without local validation).

Pillar 2 — Trend Maturity Spread

Indie restaurants in each neighborhood are classified into two pools: **competitive** (lunch-open, \$-\$\$, what your customer is choosing instead today) and **leading** (chef-driven, dinner-only or \$\$\$+, what the chef scene is popularizing for ~2 years out). Chains are excluded from both. The gap between pools is a structural proxy for trend maturity: **Rising** → **Established** → **Peak**. One flavor across three cities can show three different maturity stages — Mission al pastor is Established (ship), Williamsburg al pastor is Rising (watch), WeHo al pastor is Established but thin.

Pillar 3 — Innovation Feasibility Intelligence

Every recommendation passes through a deterministic pantry deliverability check + lift classifier. Low lift = seasoning/sauce remix from existing or shelf-stable additions. Medium = prep workflow change

or new shelf-stable SKU. High = refrigerated/frozen SKU or new cooking method. Rollout portability marks whether SKUs ship nationally or only through regional sourcing. LTO history (newsroom-verified, 20 entries across the 3 brands) downgrades lift when a brand has shipped the flavor before — supply chain dormant, not new.

Hero demonstration: Mission Mole + Chipotle gap-fill

*Mole is the strongest mover in Mission this quarter — +7 mentions Q4 2025 → Q1 2026, joining an already-Peak signal of 132 indie reviews across 12 Mission restaurants. None of the fast-casual chains (Chipotle, CAVA, Sweetgreen) can currently deliver mole from pantry. The strategic call: **Chipotle should add mole as a shelf-stable SKU and test it Mission-first.** Californios (verified comp in our scrape) sets the prestige bar; line operation is heat-and-hold in the existing steam-table well.*

The hero gap-fill card carries:

Layer	Value	Source
Composite confidence	68 / 100	scripts/compute_confidence.py
Maturity	Peak (high local volume, low chef-driven differentiation — mainstream mole culture in Mission)	scripts/compute_lift.py
Operational lift	Low + National rollout — single shelf-stable SKU addition; ships in existing line slots	data/sku_lift_classifications.json
Missing SKU	mole sauce (shelf-stable, sourceable through standard fast-casual distributors)	data/pantry_fit.json
Verified comp restaurant	Californios (24th & Mission)	data/comp_restaurant_audit.json
Indie evidence	132 total mentions, 12 restaurants, +7 vs prior quarter	data/hermes.db (Bright Data Google Maps Reviews Dataset)

The al pastor validation moment

The al pastor recommendation that ran in earlier versions of this pitch is now retired as the hero because **Chipotle relaunched Chicken Al Pastor nationally on Feb 10, 2026** — a Mission-specific recommendation reads as "do what you're already doing." But al pastor remains the strongest *validation* moment we have:

*Between Q4 2025 and Q1 2026, **al pastor** crossed from **Weak to Established** in Mission indie reviews. The maturity classifier flipped weeks before Chipotle's Jan 27 press release announcing the Feb relaunch. The system caught the structural shift in real time, before the chain announced. Mole is the next call.*

This is the durable pitch — Taste Engine catches maturity transitions that precede brand decisions. Al pastor proves the pattern works. Mole is the call the system is making right now.

Validation: does the system catch real trends?

National Google Trends for "al pastor" was a slow climb 2019–2022 (28 → 35 baseline). Chipotle's March 2023 launch of Chicken Al Pastor drove the national signal to a 7-year high (89 max, 2023 avg 51.3) and sustained it through 2024 (avg 48.5). When Chipotle withdrew in 2025 the national signal fell back near baseline (37.5) — **but Mission indie restaurants kept building**. Indie al pastor mentions in Mission climbed quarterly from 1 (2024-Q2) to 10 (2026-Q1), unaffected by the national falloff.

Taste Engine reads this as structural demand, not chain-dependent buzz. Chipotle's Feb 10, 2026 relaunch validates that reading. **The system would have flagged this in Q4 2025 from local data alone.**

Honest limit: our chronological indie review window begins 2024-Q2 — we cannot show indie buzz *leading* the March 2023 chain launch. We can show indie sustaining through 2025 and predicting the 2026 return. The validation path forward is pre-2024 reviews via Bright Data `days_limit=1095+`; documented but budget-deferred.

Bright Data dependency

Taste Engine is not reproducible from CSV exports or single-script scraping. The intelligence layer depends on Bright Data infrastructure throughout:

Bright Data product	Role in Taste Engine	Volume pulled
Google Maps Reviews Dataset (<code>gd_luzfs1dn2oa0teb81</code>)	Chronological dated reviews for indie restaurants across 3 neighborhoods	64,372 dated reviews across 383 restaurants (2024-2026, two pulls: <code>days_limit=365</code> + <code>days_limit=730</code>)
Google Maps Fast Maps Search (<code>gd_m8ebnr0q2qlklc02fz</code>)	Restaurant discovery + top_reviews per neighborhood center	383 restaurants + 4,170 embedded top-reviews
SERP API	Brand LTO press-release verification, chain timing data	20 newsroom-verified LTO entries across 3 brands
Scraping Browser	Yelp baseline reviews + ad-hoc protected-target access	239 baseline reviews + verification pulls
REST <code>/request</code> API + Proxy	All triggered via vault-managed token; local AdGuard sinkhole workaround using <code>curl --resolve</code> via Cloudflare DNS	All data acquisition

Total Bright Data spend during build: ~\$170. The chronological review dataset alone was the single largest cost (~\$80) and the largest single source of analytic value — every maturity claim, every quarterly mention chart, every validation arc derives from it.

A reproduction of this project on standard scraping infrastructure would face: Yelp DataDome blocking (we confirmed: `/search?find_loc=` pages return 1.7KB challenge HTML via Scraping Browser, replaced with Google Maps Fast Maps Search); per-place API rate limits on Google Maps Places API that make 64k chronological reviews economically infeasible; no clean path to 365-day or 730-day server-side recency filtering. Bright Data is the dependency.

Why this is hard (engineering narrative)

Headline metric: we catch eight distinct LLM hallucination classes deterministically, in code, before the model ships output. Naive LLM prompting catches zero.

That's not a tagline — it's testable. Run `scripts/naive_llm_baseline.py` and you get nine hallucinated SKUs across two dishes, fabricated supply-chain claims, and a recommendation that misses the actual brand decision happening in real life. Run `scripts/dish_generator_agent.py` and the LLM cannot assert any factual claim without first calling a tool that returns ground-truth data. Every output is gated on eight deterministic audits before a card ever appears.

The eight hallucination classes we gate

#	Class	Caught by	What it prevents
1	Pantry hallucination — recommends ingredients the brand doesn't stock	<code>scripts/compute_pantry_fit.py</code> deliverability matrix	"Cashew crema" suggested to Chipotle
2	Menu duplication — names existing dishes as novel	DO-NOT-PROPOSE prompt block populated from <code>brand_menu_items</code>	Pre-fix "Miso Salmon Bowl" sent to Sweetgreen that already sells it
3	LTO blind spot — invents from scratch what a brand already shipped	<code>data/brand_lto_history.json</code> + LTO-aware prompt	"Add quesabirria to Chipotle" (they shipped quesabrisquet; relaunch lever exists)
4	Cuisine incoherence — fuses incompatible cuisines like Japanese + Italian	<code>scripts/audit_dish_coherence.py</code> + <code>cuisine_compatibility.json</code>	"Miso Pesto Salmon Bowl"
5	Dish-name overpromise — names ingredients delivered only via marinade or substitution	<code>scripts/audit_dish_name_truthfulness.py</code> + headline-vs-background SKU tagging	"Al Pastor Pineapple Crunch Taco Plate" when pineapple is only marinade
6	Ambiguous-term false positives — non-food usages of	<code>scripts/extract_flavors.py</code> context-aware extraction	"What a jerk" review credited to jerk flavor signal

#	Class	Caught by	What it prevents
	jerk/mole/truffle inflating signal		
7	Fabricated comp restaurants — names benchmark restaurants that don't exist	<code>scripts/audit_comp_restaurants.py</code> against <code>restaurants</code> table	LLM-invented Mission landmark; we mark unverifiable refs explicitly
8	Numerical claim invention — overstates signal scores, mention counts, LTO years	<code>scripts/audit_numerical_claims.py</code> against ground-truth tables	"Backed by 200 mentions" when actual is 50

Naive LLM prompting catches none of these. Every one we've encountered in v3/v4/v5/v6/v7 iterations got fixed at the system level, not by prompt engineering — because we learned the LLM cannot be the constraint on itself.

Other load-bearing engineering

1. **Flavor normalization across paraphrases.** Reviewers say "red pepper paste" not "harissa." Vocabulary is 50 hand-curated terms; deterministic substring matching has known recall holes that we documented rather than hide. The honest framing: corroboration, not statistical significance.
2. **Dual-pool restaurant classification.** Google Maps has no "chef-driven dinner spot" field. We built a tagger combining hand-curated chain denylist + price tier + opening hours + busyness scores. Pools are mutex by construction.
3. **Cross-source corroboration weighting.** Joining geographically-coarse Google Trends with sparse paraphrased reviews on a normalized flavor entity. The composite confidence score's 6-component design (Trend 20% / Local 30% / Maturity 15% / Feasibility 20% / Recency 10% / LTO 5%) reflects multiple iterations of weighting honesty.
4. **Bright Data-scale extraction infrastructure.** 64,372 reviews required trigger → poll → snapshot-recover → dedup-by-text-hash → idempotent loaders. One snapshot's poller died mid-

download (`curl --max-time 120` truncation, 117MB JSONL); we wrote `scripts/resume_snapshot.py` to recover the Bright Data snapshot ID. We also discovered AdGuard sinkholes `*.brightdata.com` even with protection toggled off — workaround uses Cloudflare DNS resolution + `curl --resolve`.

Naive LLM vs Taste Engine — control-vs-treatment

We ran the exact same prompt — "Recommend dishes for Chipotle to launch in the Mission District based on current food trends" — against Claude Opus 4.7 with no engine context (naive control) and against Taste Engine (treatment).

The naive LLM produced **9 hallucinated SKUs across 2 dishes**: brisket, consommé, king trumpet mushrooms, cashew crema, chayote, salsa macha, garlic chips, and "beef tallow-griddled tortillas." It confidently asserted "beef brisket is already in Chipotle's supply chain" (false). It pivoted to a vegan mushroom al pastor variation, completely missing that Chipotle was already relaunching Chicken Al Pastor in real life on Feb 10, 2026 — the actual strategic opportunity.

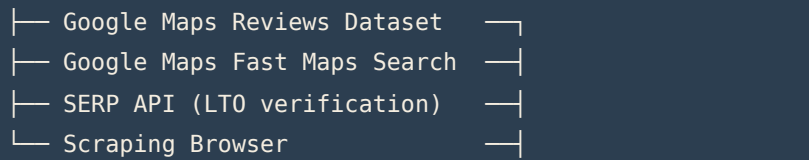
Taste Engine produced the al pastor + Chipotle hero card with every claim traceable to either the review database or a newsroom URL, plus a Mission-vs-Williamsburg rollout-targeting strategy the naive LLM cannot perform.

Full annotated comparison: `data/naive_vs_taste_engine_comparison.md`.

The pitch line: *The LLM is a reasoning layer. The innovation is the pipeline that constrains, anchors, and evidences it.*

Architecture

Bright Data



└─ Extraction + Dedup
| (text_hash idempotent loaders)

▼
SQLite (hermes.db)

├─ Flavor Vocabulary (50 terms + match_terms)
├─ Dual-Pool Tagger (indie/chain, lunch/chef-driven)
└─ Brand Pantry (Chipotle 27 + CAVA 53 + SG 160)

└─ Deliverability Matrix (per flavor × brand)

├─ Confidence Score (6 components, 0-100 composite)
├─ Operational Lift (Low/Med/High + National/Regional)
└─ LTO History (20 entries, verified URLs)

Recommendation Cards (v6)
├─ ship_now (deliverable today)
└─ gap_fill (named SKU additions)

▼
Claude Opus 4.7
(LLM = reasoning layer, not core product)

|
▼
Streamlit dashboard
Validation + Hero + Maturity
+ Signals + Pantry + Gaps

Future signal integrations

The architecture is extensible. Sources we did not build for this submission but can be added on the same infrastructure:

- **TikTok food trends** (geo-tagged hashtag mining via Bright Data)
- **Instagram geo-tagged restaurant mentions**
- **Reddit local food subreddit discussions** ([r/AskNYC](#) , [r/AskSF](#) , [r/LosAngeles](#))
- **DoorDash / Uber Eats menu change monitoring** (which dishes get added/removed from chain test markets)
- **Yelp review deltas** (via Bright Data Dataset for proper compliance)
- **Image OCR from food photography** (visual flavor signal in restaurant Instagrams)
- **Seasonal ingredient pricing** (specialty distributor APIs for SKU cost forecasting)

Each adds one column to the signal-fusion equation. None require an architectural rewrite.

Future advanced feature: Regional Rollout Simulation

Pre-launch question Taste Engine could answer: "What happens if Chipotle ships Chicken Al Pastor in Williamsburg first?"

Outputs: - Operational lift (Low — pantry already supports it) - Pantry compatibility (100% — all SKUs active) - Maturity reading (Rising in WB, Established in Mission) → **rollout sequencing recommendation** - Authenticity risk (Williamsburg indie scene has 8 chef-driven al pastor mentions vs only 3 competitive — chain version risks looking opportunistic) - Adjacent markets prediction (WeHo Established-but-thin → likely warm reception)

This is the natural next layer on top of the existing pipeline. The data is there; the simulation surface is not yet built.

Demo flow (5-minute script anchor)

1. **Cold open + what it is** (0:00–0:30) — Brands run quarterly menu planning; the bottleneck isn't creativity, it's neighborhood-level intelligence.
 2. **Bright Data dependency** (0:30–1:00) — 64,372 dated indie reviews, 383 restaurants, 3 metro markets, 24 months. Four Bright Data products composed: Google Maps Reviews Dataset, Fast Maps Search, SERP API, Scraping Browser.
 3. **Act 1 — Trend Maturity Spread** (1:00–2:00) — al pastor across 3 cities: Established in Mission, Rising in Williamsburg, Established-but-thin in WeHo. One flavor, three maturity stages.
 4. **Act 2 — Validation, then the call** (2:00–3:00) — al pastor crossed Weak → Established in Mission between Q4 2025 and Q1 2026, weeks before Chipotle's Jan 27 relaunch announcement. The system caught the structural shift before the brand made the call. Mole is the next call: hero gap-fill card.
 5. **Act 3 — Quarterly cadence + forecast** (3:00–3:45) — Mole +7 mentions Q1, birria forecast Q2 ~27 rising. Brands return monthly/quarterly for the deltas, not a one-shot analysis.
 6. **Naive LLM vs Taste Engine** (3:45–4:15) — 8 deterministic hallucination classes caught in code vs 0 caught by naive prompting. The LLM is a reasoning layer; the pipeline is the innovation.
 7. **Why this is hard + close** (4:15–4:45) — Eight constraint layers, Bright Data infrastructure, neighborhood-aware culinary intelligence built on Bright Data.
-

Stack

- **Data:** SQLite (`data/hermes.db`), JSON-derived configs, Bright Data JSONL pulls
- **Ingestion:** Python 3.12 scripts in `scripts/` (numbered roughly in dependency order)
- **LLM layer:** Claude Opus 4.7 (`claude-opus-4-7`) via Anthropic SDK with prompt caching (~70% input savings on city re-runs)

- **Dashboard:** Streamlit with Altair charts, custom CSS
- **Trend data:** pytrends (patched for urllib3≥2.0 compatibility)
- **Vault:** Wundervault for ANTHROPIC_API_KEY + Bright Data tokens

Run locally

```
cd taste-engine
set -a && source .env && set +a
.venv/bin/streamlit run app.py --server.headless true
# → http://localhost:8501
```

Full regeneration pipeline documented in [HANDOFF.md](#) (no Bright Data spend needed if [data/hermes.db](#) is intact).

License + acknowledgments

Built for the Bright Data hackathon, May 2026. All Bright Data product references are factual descriptions of products used in the project.